

Obsidian + Claude Code

打造个人知识库

从工具到方法论，构建你的第二大脑

部门内部知识分享 · 2026 年 7 月

今天聊什么

		
第一部分	第二部分	第三部分
Obsidian 入门	LLM Wiki 范式	知识库网站部署
软件介绍 + 核心插件	Claude Code 驱动的知识管理	GitHub Pages 自动上线

第一部分

 Obsidian: 你的第二大脑

Obsidian 是什么

本地优先、基于 *Markdown* 的个人知识管理工具

所有笔记以 `.md` 纯文本保存在你的硬盘上，通过 `[[双向链接]]` 编织知识网络。

核心理念：File over App

数据永远属于你。即使 *Obsidian* 公司消失，任何文本编辑器都能打开你的笔记。

几个惊人的数字

团队

- 7 人 + 1 只猫，工程团队仅 3 人
- 零外部融资，估值 3.5 亿美元
- 100% 靠用户付费

生态

- 2700+ 社区插件
- 24 万+ 周活跃 Reddit 用户
- 2020 年 3 月诞生于疫情隔离

哲学

- 不做强制闭源 AI
- 不让资本绑架产品
- 唯一的老板 = 用户

核心特性一览

特 性	说 明	为 什 么 重 要
📄 本地 Markdown	文件在你硬盘上	数据主权、可移植、Git 友好
🔗 双向链接	[[笔记名]] 自动关联	网状思维，自下而上组织
🗺️ 知识图谱	可视化笔记网络	发现隐藏关联
🔧 插件系统	2700+ 开源插件	按需扩展，不臃肿
🎨 高度可定制	主题、CSS 片段	打造专属工作环境

Markdown: AI 时代的"通用语"

“

三个独立十亿级项目 (*Manus*、*OpenClaw*、*Claude Code*) 殊途同归, 都选择 *Markdown* 作为 *Agent* 记忆层

”

- **Token 效率**: 比 JSON/XML 高 30-50%
- **LLM 原生理解**: 模型在 Markdown 上训练最多
- **人类可读**: AI 产出你也能直接看懂
- **工具链成熟**: Git diff、静态站点生成、批量处理

必备插件推荐

类别	插件	下载量	解决什么问题
 同步	Git	230万+	免费多端同步 + 版本历史
 查询	Dataview	390万+	笔记数据库，类 SQL 查询
 模板	Templater	390万+	动态模板，自动填充日期 / 标题
 日历	Calendar	250万+	日记导航，习惯追踪
 捕获	QuickAdd	—	2 秒捕获想法
 图片	Custom Attachment Location	—	图片按笔记自动归类
 导出	Enhancing Export	—	多格式导出

重点：Git 插件 — 免费云同步

为什么选 Git + GitHub?

vs	Git + GitHub	iCloud/OneDrive	Obsidian Sync
💰 费用	 免费	 免费	\$4 / 月
📖 版本历史	✅ 完整 Git 历史	❌ 有限	✅ 有
⏮ 回滚	✅ 任意版本	❌ 困难	✅ 有
🔒 隐私	✅ 私有仓库	⚠ 云端	✅ 端到端加密

Git 插件配置 (5 分钟搞定)

1. GitHub 创建私有仓库 → 2. 克隆到本地 → 3. Obsidian 打开

↓

4. 创建 .gitignore
.obsidian/workspace
.trash/

5. 安装 Git 插件
作者 Denis Olekhov

6. 配置自动同步
停止编辑 1 分钟后
自动 commit + push

核心配置	推荐值
Auto commit-and-sync interval	1 分钟
Pull on startup	开启
Push on commit-and-sync	开启

配置完就不用管了，状态栏会自动显示同步状态 🍪

多重备份策略

- 🔥 热备份 — GitHub 私有仓库（实时同步 · 多端可用）
 - 🌡️ 温备份 — iCloud / 百度云（每周自动备份）
 - ❄️ 冷备份 — 移动硬盘 / U盘（每月手动备份）

即使 *GitHub* 关闭、电脑损坏——你的笔记依然安全。

第二部分



Claude Code 驱动的 LLM Wiki

Claude Code 是什么

Anthropic 推出的终端 AI 编程 Agent

比普通 AI Chat 多了一层 "规划 + 自主执行" 能力：

能力	说明
 文件操作	读、写、编辑项目中的所有文件
 命令执行	运行 shell 命令、脚本、测试
 Git 管理	自动 commit、PR、分支管理
 MCP 扩展	连接浏览器、数据库、外部 API
 Skills 系统	可安装的方法论包，AI 自主调用

核心思想：AI 不是检索器，是编译器

Karpathy 2026 年 4 月提出，1600 万浏览



lint = 测试 (发现矛盾) · query = 运行时 (提问)

三层架构



操作一：Ingest（收录）= 编译

丢一篇文章进 `raw/`，AI 自动完成：

1. 提取关键信息
2. 创建或更新**概念页**（新概念）和**实体页**（新工具/人物）
3. 更新相关**专题页**
4. 更新全局概览 + 索引 + 操作日志
5. 检查教程是否需要同步
6. 重建网站数据

一篇资料可能触达 10-15 个 wiki 页面！

操作二 & 三：Query + Lint

 Query（查询）= 运行时

提问 → AI 先读索引定位 → 精读相关页 → 综合回答 + `[[引用]]`

原则：先搜知识库，不够才联网。

 Lint（巡检）= 测试

定期健康检查：

- 页面矛盾？ → 标注 `⚠ 矛盾`
- 过时论断？ → 标记需更新
- 孤立页面？ → 补充入链
- 缺失引用？ → 补充交叉链接

CLAUDE.md: AI 的"宪法"

```
# 知识库 Schema

## 基本规则
- 所有回复使用中文
- 先搜知识库，不够才联网
- 联网后有新信息，主动询问是否收录

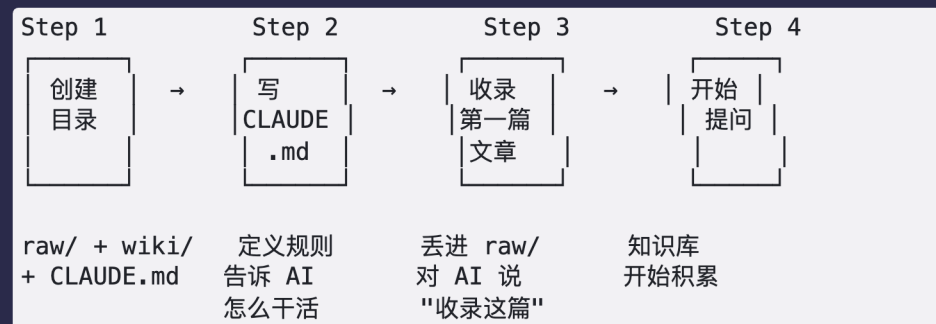
## 目录结构: raw/ → wiki/ → output/

## 页面规范
- YAML frontmatter (title, type, tags, sources, related)
- 命名: 英文小写 + 连字符 · 标题: 中文

## 工作流: 1. Ingest 2. Query 3. Lint 4. Output
```

护栏，不是手册。每次 AI 犯错就加一条规则——文件是活的，一直在长。

实践：从零开始的 4 步



一个最小但持续使用的 LLM Wiki，比一个用向量数据库但没人维护的 RAG 系统强一万倍。

LLM Wiki vs 传统 RAG

维度	传统 RAG	LLM Wiki
🧠 知识状态	无状态，每次从零发现	有状态，持续积累
🔧 基础设施	向量数据库 + Embedding	Git + Markdown
📄 知识复利	无积累	新知识建立在旧知识之上
⚠️ 发现矛盾	不检测	Lint 主动发现
🔗 交叉引用	依赖检索质量	编译时预处理
🏠 维护负担	低	中（AI 自动维护大部分）

RAG 每次查询从零发现——没有积累。LLM Wiki 知识编译一次，永久可用。

第三部分

部署个人知识库网站

网站技术栈

```
Markdown 源文件      Wiki/wiki/*.md
  ↓
Python 构建脚本      build.py (296 行)
  提取 frontmatter → Markdown → HTML → 转换 Wiki 链接
  ↓
结构化 JSON          data/data.json
  ↓
纯前端 SPA           index.html + app.js (~500 行)
  Hash 路由 · 全文搜索 · 标签云 · 明暗主题
  ↓
GitHub Pages          自动构建 + 部署
```

前端功能一览

功能	实现
🏠 首页仪表盘	统计卡片 + 最近更新 + 快速导航
📖 全部页面浏览	按类型 / 标签过滤，全文搜索
🏷️ 标签云	自动聚合，按使用频率排序
🔍 搜索 (Ctrl+K)	全量模糊搜索，即时响应
🌑 主题切换	明 / 暗模式，跟随系统或手动切换
📖 教程阅读器	上下篇导航，完整渲染
📱 响应式设计	手机 / 平板 / 桌面全适配

GitHub Actions 自动部署

触发条件: push 到 main 分支 + Wiki 文件变更

Checkout → 构建数据(build.py) → 验证产物(data.json) → 部署到 GitHub Pages

每次更新 Wiki → 自动构建 → 自动部署 → 网站实时同步

关键配置

- 触发路径: Wiki/wiki/**、Wiki/raw/**、Wiki/site/**
- Python 版本: 3.12 · 部署目标: GitHub Pages (免费)
- 在线地址: <https://<用户名>.github.io/<仓库名>/>

网站项目结构

```
Wiki/site/
├── index.html      ← SPA 入口 (100 行)
├── css/style.css   ← 学者书房美学 (宋体+铜金)
├── js/app.js       ← 路由 + 搜索 + 渲染 (~400 行)
├── data/data.json  ← 构建产物 (所有页面结构化数据)
├── build.py        ← Python 构建脚本 (296 行)
└── requirements.txt ← pyyaml + markdown
```

设计理念： 宋体标题 + 铜金点缀 · 暖纸墨色阅读体验 · 零后端纯静态

你也可以这样做

准备工作

- 1. 一个 GitHub 账号 (免费)
- 2. Obsidian Vault 推送到 Git 仓库
- 3. 写一份 CLAUDE.md (定义规则)
- 4. 配置 .github/workflows/deploy.yml

成本分析

项目	费用
Obsidian 个人使用	 免费
GitHub 私有仓库	 免费
GitHub Pages 托管	 免费
GitHub Actions 构建	 免费 (2000 分钟/月)
Claude Code API	 按量付费 (可接国产模型降成本)

总结

三个关键词

Obsidian

数据主权 + 本地 Markdown

- 双向链接 + 插件生态

你的第二大脑

Claude Code

LLM Wiki 范式 + 三层架构

- Ingest/Query/Lint

AI 驱动的知识编译器

网站部署

Python 构建 + 纯前端 SPA

- GitHub Pages

知识库即网站，一键上线

核心金句

"AI 不是检索器，是编译器。" —— Karpathy

"数据永远属于你。" —— Obsidian 哲学

"一个最小但持续使用的 LLM Wiki，比一个用向量数据库但没人维护的 RAG 系统强一万倍。"

"护栏，不是手册。" —— CLAUDE.md 设计原则

行动起来

今天就试试：

1. 下载 Obsidian, 建一个 Vault
2. 装 Git 插件, 推到 GitHub 私有仓库
3. 装 Claude Code: `npm install -g @anthropic-ai/claude-code`
4. 创建 `raw/` + `wiki/` + `CLAUDE.md`
5. 扔一篇文章进 `raw/`, 对 AI 说: "帮我收录这篇文章"

Q & A

谢谢大家！

 本知识库在线地址：[Wiki Vault](#)